

APLIKASI CHAT MULTI-USER NETHUB DENGAN IMPLEMENTASI TCP SOCKET DAN FTP SERVER

LAPORAN TUGAS AKHIR

Diajukan untuk Memenuhi Kelulusan Mata Kuliah Network Programming



DISUSUN OLEH :

714240008 MUHAMMAD ARIF RIVALDI

DOSEN PENGAMPU :

M. Yusril Helmi Setyawan, S.Kom., M.Kom., SFPC.

**PROGRAM STUDI DIV TEKNIK INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS LOGISTIK DAN BISNIS INTERNASIONAL
BANDUNG**

2026

DAFTAR ISI

DAFTAR ISI	i
DAFTAR GAMBAR	iv
DAFTAR TABEL	v
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan	2
1.4 Manfaat	2
1.5 Batasan Masalah	2
BAB 2 LANDASAN TEORI	4
2.1 Network Programming	4
2.2 Socket Programming	4
2.2.1 Jenis Socket	4
2.2.2 Socket Programming di Python	5
2.3 Protokol TCP (Transmission Control Protocol)	5
2.3.1 Three-Way Handshake	5
2.4 File Transfer Protocol (FTP)	6
2.4.1 Karakteristik FTP	6
2.4.2 Library pyftplib	6
2.5 Threading dan Concurrency	6
2.5.1 Threading di Python	7
2.5.2 Thread Safety	7
2.6 Library Python yang Digunakan	8
2.7 JSON (JavaScript Object Notation)	8
BAB 3 PERANCANGAN DAN IMPLEMENTASI	9
3.1 Arsitektur Sistem	9

3.2	Perancangan Database	10
3.2.1	Struktur users.json	10
3.2.2	Struktur files_metadata.json	11
3.3	Implementasi Server (server.py)	11
3.3.1	Inisialisasi Server	11
3.3.2	Multi-threading untuk Multiple Client	12
3.3.3	Protokol Komunikasi	12
3.3.4	Mekanisme Broadcast	13
3.3.5	Autentikasi Pengguna	13
3.4	Implementasi Client GUI (client_gui.py)	13
3.4.1	Struktur Class NetHubApp	14
3.4.2	Fitur-Fitur Client	14
3.4.3	Receiving Messages	15
3.5	Implementasi FTP Server (ftp_server.py)	15
3.5.1	Custom Authorizer	15
3.5.2	Struktur Direktori FTP	16
3.5.3	Konfigurasi FTP Server	16
3.6	Implementasi FTP Browser (ftp_browser.py)	16
3.6.1	Fitur FTP Browser	17
3.6.2	Koneksi FTP	17
3.7	Implementasi Game Multiplayer (game_window.py)	18
3.7.1	Logika Game	18
3.7.2	Sinkronisasi State Game	18
BAB 4	PENGUJIAN	19
4.1	Pengujian Koneksi Server-Client	19
4.1.1	Skenario Pengujian	19
4.1.2	Hasil Pengujian	19
4.2	Pengujian Autentikasi	19
4.2.1	Pengujian Register	19
4.2.2	Pengujian Login	20
4.3	Pengujian Fitur Chat	20
4.3.1	Pengujian Room System	20
4.3.2	Pengujian Messaging	20
4.4	Pengujian Transfer File	21
4.4.1	Pengujian Upload File	21
4.4.2	Pengujian Download File	21
4.5	Pengujian FTP Server	21

4.5.1	Pengujian Koneksi FTP	21
4.5.2	Pengujian Operasi File FTP	21
4.6	Pengujian Game Multiplayer	22
4.6.1	Pengujian Tic-Tac-Toe	22
4.7	Pengujian Speed Test	22
4.8	Ringkasan Hasil Pengujian	22
BAB 5	KESIMPULAN DAN SARAN	23
5.1	Kesimpulan	23
5.2	Saran	24

DAFTAR GAMBAR

3.1	Diagram Alur Kerja Sistem NetHub	9
-----	--	---

DAFTAR TABEL

2.1	Library Python yang Digunakan	8
3.1	Protokol Komunikasi NetHub	12
4.1	Hasil Pengujian Koneksi Server-Client	19
4.2	Hasil Pengujian Register	19
4.3	Hasil Pengujian Login	20
4.4	Hasil Pengujian Room System	20
4.5	Hasil Pengujian Messaging	20
4.6	Hasil Pengujian Upload File	21
4.7	Hasil Pengujian Download File	21
4.8	Hasil Pengujian Koneksi FTP	21
4.9	Hasil Pengujian Operasi File FTP	21
4.10	Hasil Pengujian Game Tic-Tac-Toe	22
4.11	Hasil Pengujian Speed Test	22

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi yang pesat telah mengubah cara manusia berinteraksi dan berkomunikasi. Salah satu bentuk komunikasi digital yang paling umum digunakan adalah aplikasi chat atau pesan instan. Aplikasi chat memungkinkan pengguna untuk berkomunikasi secara real-time tanpa batasan waktu dan jarak geografis.

Dalam konteks pembelajaran *Network Programming*, pemahaman tentang bagaimana aplikasi jaringan bekerja sangatlah penting. Konsep-konsep seperti *socket programming*, protokol TCP/IP, arsitektur *client-server*, dan *multi-threading* merupakan fondasi dasar dalam pengembangan aplikasi berbasis jaringan.

NetHub dikembangkan sebagai implementasi praktis dari konsep-konsep *network programming*. Aplikasi ini tidak hanya menyediakan fitur chat sederhana, tetapi juga mengintegrasikan berbagai fitur tambahan seperti *File Transfer Protocol* (FTP), permainan multiplayer, dan pengukuran kecepatan jaringan. Dengan demikian, NetHub menjadi platform pembelajaran yang komprehensif untuk memahami berbagai aspek pemrograman jaringan.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam laporan ini adalah:

1. Bagaimana mengimplementasikan aplikasi chat multi-user menggunakan TCP socket di Python?
2. Bagaimana mengintegrasikan sistem autentikasi pengguna dalam aplikasi chat?
3. Bagaimana mengimplementasikan fitur transfer file dalam aplikasi chat?
4. Bagaimana mengintegrasikan FTP server dengan sistem autentikasi yang sama?

5. Bagaimana mengimplementasikan game multiplayer dalam aplikasi chat berbasis jaringan?

1.3 Tujuan

Tujuan dari pembuatan aplikasi NetHub ini adalah:

1. Mengimplementasikan aplikasi chat multi-user menggunakan TCP socket dengan Python.
2. Membangun sistem autentikasi pengguna dengan penyimpanan data berbasis JSON.
3. Mengimplementasikan fitur upload dan download file dalam chat.
4. Mengintegrasikan FTP server yang terintegrasi dengan sistem autentikasi utama.
5. Mengembangkan fitur game multiplayer Tic-Tac-Toe berbasis jaringan.
6. Menyediakan fitur pengukuran kecepatan jaringan (speed test).

1.4 Manfaat

Manfaat dari pengembangan aplikasi NetHub adalah:

1. **Manfaat Akademis:** Sebagai sarana pembelajaran praktis tentang konsep-konsep *network programming* seperti *socket programming*, protokol TCP/IP, dan arsitektur *client-server*.
2. **Manfaat Praktis:** Menyediakan aplikasi chat yang dapat digunakan untuk komunikasi dalam jaringan lokal dengan berbagai fitur tambahan.
3. **Manfaat Pengembangan:** Sebagai dasar pengembangan aplikasi jaringan yang lebih kompleks di masa depan.

1.5 Batasan Masalah

Batasan masalah dalam pengembangan aplikasi NetHub adalah:

1. Aplikasi berjalan pada jaringan lokal (localhost) dengan alamat IP 127.0.0.1.
2. Server chat berjalan pada port 55555 dan FTP server pada port 2121.

3. Antarmuka pengguna menggunakan Tkinter (GUI) dan Command Line Interface (CLI).
4. Data pengguna disimpan dalam format JSON.
5. Tidak ada enkripsi end-to-end pada komunikasi chat.

BAB 2

LANDASAN TEORI

2.1 Network Programming

Network Programming adalah cabang pemrograman yang berfokus pada penulisan program yang berkomunikasi melalui jaringan komputer. Pemrograman jaringan melibatkan penggunaan protokol komunikasi seperti TCP/IP, UDP, HTTP, FTP, dan lainnya untuk mengirim dan menerima data antar komputer yang terhubung dalam jaringan.

Dalam konteks *network programming*, terdapat dua model arsitektur utama:

1. **Client-Server Architecture:** Model di mana terdapat server yang menyediakan layanan dan client yang mengakses layanan tersebut.
2. **Peer-to-Peer (P2P) Architecture:** Model di mana setiap node dapat bertindak sebagai client sekaligus server.

2.2 Socket Programming

Socket adalah titik akhir (*endpoint*) dari komunikasi dua arah antara dua program yang berjalan di jaringan. Socket merupakan kombinasi dari alamat IP dan nomor port yang mengidentifikasi koneksi jaringan secara unik.

2.2.1 Jenis Socket

Terdapat dua jenis socket utama dalam pemrograman jaringan:

1. **Stream Socket (SOCK_STREAM):** Menggunakan protokol TCP yang menjamin pengiriman data secara berurutan dan andal.
2. **Datagram Socket (SOCK_DGRAM):** Menggunakan protokol UDP yang lebih cepat tetapi tidak menjamin urutan dan keandalan pengiriman.

2.2.2 Socket Programming di Python

Python menyediakan modul `socket` untuk implementasi *socket programming*. Berikut adalah operasi dasar socket:

```
1 import socket
2
3 # Membuat socket TCP
4 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5
6 # Binding ke alamat dan port
7 server.bind(('127.0.0.1', 5555))
8
9 # Listening untuk koneksi
10 server.listen()
11
12 # Menerima koneksi dari client
13 client, addr = server.accept()
```

Listing 2.1: Contoh Dasar Socket Server

2.3 Protokol TCP (Transmission Control Protocol)

TCP adalah protokol komunikasi yang berorientasi koneksi (*connection-oriented*) pada layer transport dalam model OSI. Karakteristik utama TCP meliputi:

1. **Reliable:** Menjamin data sampai ke tujuan tanpa error.
2. **Ordered:** Data diterima dalam urutan yang benar.
3. **Error-checked:** Menggunakan checksum untuk mendeteksi error.
4. **Flow Control:** Mengatur kecepatan pengiriman data.
5. **Congestion Control:** Menghindari kemacetan jaringan.

2.3.1 Three-Way Handshake

Proses pembentukan koneksi TCP menggunakan mekanisme *three-way handshake*:

1. **SYN:** Client mengirim paket SYN ke server.
2. **SYN-ACK:** Server membalas dengan paket SYN-ACK.
3. **ACK:** Client mengirim paket ACK untuk mengkonfirmasi koneksi.

2.4 File Transfer Protocol (FTP)

FTP adalah protokol jaringan standar yang digunakan untuk mentransfer file antara client dan server melalui jaringan TCP/IP. FTP beroperasi pada port 21 untuk kontrol dan port 20 untuk transfer data.

2.4.1 Karakteristik FTP

1. Menggunakan dua koneksi terpisah: kontrol dan data.
2. Mendukung mode aktif dan pasif untuk transfer data.
3. Menyediakan autentikasi menggunakan username dan password.
4. Mendukung berbagai operasi file: upload, download, delete, rename.

2.4.2 Library pyftplib

`pyftplib` adalah library Python yang menyediakan implementasi FTP server yang lengkap dan dapat dikustomisasi. Library ini mendukung:

- Autentikasi pengguna kustom
- Manajemen direktori per pengguna
- Rate limiting
- SSL/TLS untuk koneksi aman

2.5 Threading dan Concurrency

Threading adalah teknik untuk menjalankan beberapa tugas secara bersamaan dalam satu proses. Dalam konteks server aplikasi, threading digunakan untuk menangani multiple client secara simultan.

2.5.1 Threading di Python

Python menyediakan modul `threading` untuk implementasi multi-threading:

```
1 import threading
2
3 def handle_client(client):
4     # Menangani client
5     pass
6
7 # Membuat thread baru untuk setiap client
8 thread = threading.Thread(target=handle_client, args=(client,))
9 thread.start()
```

Listing 2.2: Contoh Multi-threading

2.5.2 Thread Safety

Ketika multiple thread mengakses data bersama, diperlukan mekanisme sinkronisasi untuk mencegah *race condition*:

```
1 import threading
2
3 lock = threading.Lock()
4
5 with lock:
6     # Operasi pada shared data
7     pass
```

Listing 2.3: Penggunaan Lock

2.6 Library Python yang Digunakan

Berikut adalah library Python yang digunakan dalam pengembangan NetHub:

Table 2.1: Library Python yang Digunakan

Library	Fungsi
socket	Implementasi socket programming untuk komunikasi jaringan
threading	Multi-threading untuk menangani multiple client
tkinter	Graphical User Interface (GUI) untuk aplikasi client
pyftplib	Implementasi FTP server
json	Parsing dan serialisasi data JSON
hashlib	Hashing password menggunakan SHA-256
base64	Encoding file untuk transfer melalui socket
speedtest-cli	Pengukuran kecepatan internet

2.7 JSON (JavaScript Object Notation)

JSON adalah format pertukaran data yang ringan dan mudah dibaca. Dalam NetHub, JSON digunakan untuk:

1. Menyimpan data pengguna (`users.json`)
2. Menyimpan metadata file (`files_metadata.json`)
3. Protokol komunikasi antar client dan server

Contoh struktur data JSON untuk pengguna:

```
1 {  
2     "username": {  
3         "password": "hashed_password",  
4         "role": "user"  
5     }  
6 }
```

Listing 2.4: Struktur `users.json`

BAB 3

PERANCANGAN DAN IMPLEMENTASI

3.1 Arsitektur Sistem

NetHub menggunakan arsitektur *client-server* di mana terdapat satu server pusat yang melayani multiple client. Server bertanggung jawab untuk mengelola koneksi, autentikasi, dan routing pesan antar client dalam room yang sama.

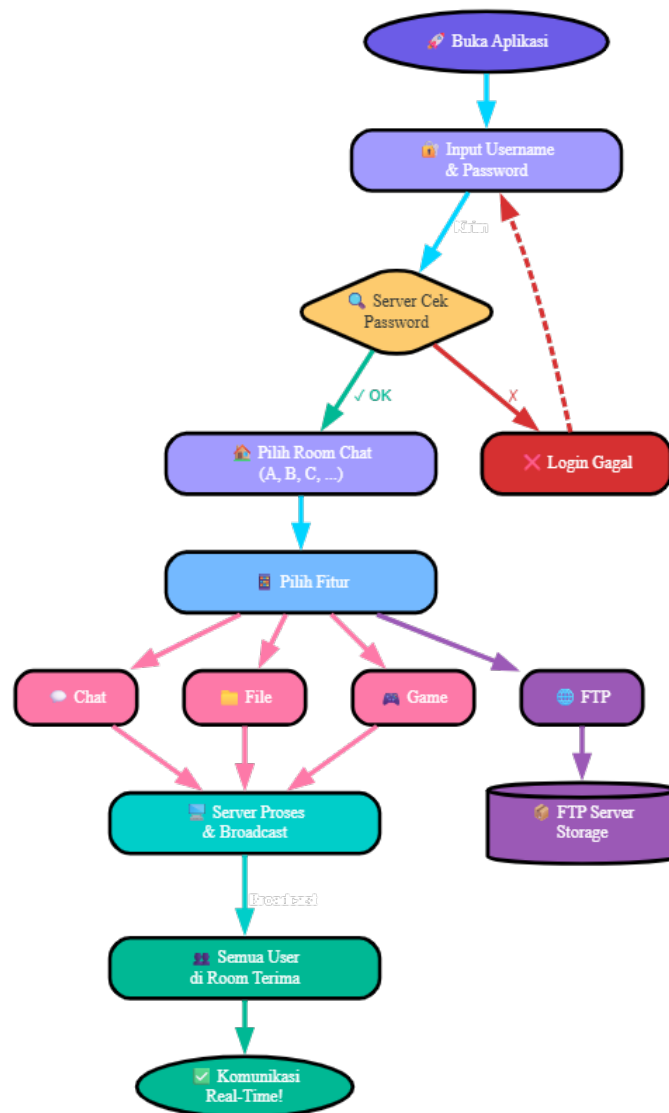


Figure 3.1: Diagram Alur Kerja Sistem NetHub

Berdasarkan Gambar 3.1, alur kerja sistem NetHub adalah sebagai berikut:

1. Pengguna membuka aplikasi dan memasukkan username serta password
2. Server melakukan validasi kredensial terhadap database JSON
3. Jika berhasil, pengguna memilih room chat yang diinginkan
4. Dalam room, pengguna dapat memilih fitur: Chat, File, Game, atau FTP
5. Untuk fitur Chat, File, dan Game, server memproses dan melakukan broadcast ke semua user di room yang sama
6. Untuk fitur FTP, pengguna terhubung ke FTP Server terpisah yang memiliki storage tersendiri

3.2 Perancangan Database

NetHub menggunakan file JSON sebagai penyimpanan data. Terdapat dua file utama:

3.2.1 Struktur users.json

File `users.json` menyimpan data pengguna dengan format sebagai berikut:

```
1 {  
2   "admin": {  
3     "password": "8c6976e5b5410415bde908bd4dee15dfb16...",  
4     "role": "admin"  
5   },  
6   "user1": {  
7     "password": "ef92b778bafef771e89245b89ecbc08a44a...",  
8     "role": "user"  
9   }  
10 }
```

Listing 3.1: Struktur Data users.json

Password disimpan dalam bentuk hash SHA-256 untuk keamanan.

3.2.2 Struktur files_metadata.json

File `files_metadata.json` menyimpan informasi file yang diupload beserta room asalnya:

```
1 {  
2     "document.pdf": "A-1",  
3     "image.png": "B-2"  
4 }
```

Listing 3.2: Struktur Data `files_metadata.json`

3.3 Implementasi Server (`server.py`)

Server merupakan komponen inti dari sistem NetHub. Server bertanggung jawab untuk:

1. Menerima dan mengelola koneksi client
2. Autentikasi pengguna (login dan register)
3. Routing pesan ke room yang sesuai
4. Mengelola file upload dan download
5. Broadcasting game state

3.3.1 Inisialisasi Server

```
1 import socket  
2 import threading  
3  
4 HOST = '127.0.0.1'  
5 PORT = 55555  
6  
7 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
8 server.bind((HOST, PORT))  
9 server.listen()
```

Listing 3.3: Inisialisasi TCP Socket Server

Server menggunakan socket TCP yang di-bind ke alamat localhost dengan port 55555.

3.3.2 Multi-threading untuk Multiple Client

Setiap client yang terhubung ditangani oleh thread terpisah:

```
1 def receive():
2     print(f"Server is listening on {HOST}:{PORT}")
3     while True:
4         client, address = server.accept()
5         print(f"Connected with {str(address)}")
6         thread = threading.Thread(target=handle_client,
7                                   args=(client, address))
8         thread.start()
```

Listing 3.4: Thread Handling untuk Client

3.3.3 Protokol Komunikasi

NetHub menggunakan protokol berbasis teks dengan format:

COMMAND | PARAMETER1 | PARAMETER2 | . . .

Tabel 3.1 menjelaskan perintah-perintah yang digunakan:

Table 3.1: Protokol Komunikasi NetHub

Command	Deskripsi
REGISTER user pass	Mendaftarkan pengguna baru
LOGIN user pass	Login ke sistem
JOIN_ROOM room	Bergabung ke room tertentu
MSG content	Mengirim pesan chat
UPLOAD filename data	Upload file (base64 encoded)
DOWNLOAD filename	Request download file
GAME action params	Perintah game multiplayer

3.3.4 Mekanisme Broadcast

Fungsi broadcast mengirim pesan ke semua client dalam room yang sama:

```
1 def broadcast_to_room(room, message):
2     with lock:
3         for client_sock, data in clients.items():
4             if data.get('room') == room:
5                 try:
6                     client_sock.send((message + "\n").encode('utf
7 -8'))
8                 except:
9                     # Handle disconnected client
10                    pass
```

Listing 3.5: Fungsi Broadcast ke Room

3.3.5 Autentikasi Pengguna

Sistem autentikasi menggunakan hashing SHA-256:

```
1 import hashlib
2
3 def hash_password(password):
4     return hashlib.sha256(password.encode()).hexdigest()
5
6 def login_user(username, password):
7     users = load_users()
8     if username not in users:
9         return False, None
10
11     hashed = hash_password(password)
12     stored_pass = users[username]["password"]
13     role = users[username]["role"]
14
15     if hashed == stored_pass:
16         return True, role
17     return False, None
```

Listing 3.6: Fungsi Hash Password

3.4 Implementasi Client GUI (client_gui.py)

Client GUI dibangun menggunakan library Tkinter dengan arsitektur berbasis frame untuk navigasi antar layar.

3.4.1 Struktur Class NetHubApp

```
1 class NetHubApp:
2     def __init__(self):
3         self.client = socket.socket(socket.AF_INET, socket.
4         SOCK_STREAM)
5         self.connected = False
6         try:
7             self.client.connect((HOST, PORT))
8             self.connected = True
9         except:
10            messagebox.showerror("Error", "Could not connect to
11            server")
12
13            # Initialize Tkinter window
14            self.root = tk.Tk()
15            self.root.title("NetHub")
16            # ... UI initialization
```

Listing 3.7: Struktur Class NetHubApp

3.4.2 Fitur-Fitur Client

Client GUI menyediakan beberapa fitur utama:

1. **Login/Register:** Form untuk autentikasi pengguna
2. **Room Selection:** Dropdown untuk memilih room chat
3. **Chat Interface:** Area untuk mengirim dan menerima pesan
4. **File Upload:** Button untuk upload file ke server
5. **FTP Browser:** Window untuk mengakses FTP storage
6. **Game:** Window untuk bermain Tic-Tac-Toe
7. **Speed Test:** Pengukuran kecepatan internet

3.4.3 Receiving Messages

Client menggunakan thread terpisah untuk menerima pesan dari server:

```
1 def receive(self):
2     while True:
3         try:
4             message = self.buf.readline()
5             if message is None:
6                 break
7
8             parts = message.split('|')
9             cmd = parts[0]
10
11             if cmd == "MSG":
12                 user = parts[1]
13                 content = parts[2]
14                 self.add_chat(user, content)
15                 # ... handle other commands
16         except:
17             break
```

Listing 3.8: Thread untuk Menerima Pesan

3.5 Implementasi FTP Server (ftp_server.py)

FTP Server dibangun menggunakan library `pyftplib` dengan custom authorizer yang terintegrasi dengan sistem autentikasi utama.

3.5.1 Custom Authorizer

```
1 class NetHubAuthorizer(DummyAuthorizer):
2     def validate_authentication(self, username, password, handler)
3     :
4         users = load_users()
5
6         if username not in users:
7             raise AuthenticationFailed("Invalid credentials")
8
9         hashed_input = hash_password(password)
10        stored_hash = users[username]["password"]
11
12        if hashed_input != stored_hash:
13            raise AuthenticationFailed("Invalid credentials")
```

```

14         # Create user directory
15         user_dir = create_user_directory(username)
16
17         # Add user with permissions
18         if not self.has_user(username):
19             self.add_user(username, password, user_dir, perm="
elradfmw")
20
21         return True

```

Listing 3.9: Custom FTP Authorizer

3.5.2 Struktur Direktori FTP

Setiap pengguna memiliki direktori pribadi di dalam folder ftp_data:

```

ftp_data/
+-- public/          # Akses anonymous (read-only)
+-- user1/           # Direktori user1
+-- user2/           # Direktori user2
+-- ...

```

3.5.3 Konfigurasi FTP Server

```

1 FTP_HOST = '127.0.0.1'
2 FTP_PORT = 2121
3 FTP_DATA_DIR = 'ftp_data'
4
5 handler = NetHubFTPHandler
6 handler.authorizer = authorizer
7 handler.passive_ports = range(60000, 60100)
8 handler.banner = "Welcome to NetHub FTP Server"
9
10 server = FTPServer((FTP_HOST, FTP_PORT), handler)
11 server.max_cons = 50
12 server.max_cons_per_ip = 5

```

Listing 3.10: Konfigurasi FTP Server

3.6 Implementasi FTP Browser (ftp_browser.py)

FTP Browser menyediakan antarmuka GUI untuk mengakses FTP server.

3.6.1 Fitur FTP Browser

1. **Browse:** Menelusuri direktori FTP
2. **Upload:** Mengunggah file ke FTP server
3. **Download:** Mengunduh file dari FTP server
4. **Delete:** Menghapus file dari FTP server
5. **Share to Chat:** Membagikan file ke room chat

3.6.2 Koneksi FTP

```
1 from ftplib import FTP
2
3 def _connect(self):
4     def do_connect():
5         try:
6             self.ftp = FTP()
7             self.ftp.connect(self.host, self.port)
8             self.ftp.login(self.username, self.password)
9             self._refresh()
10        except Exception as e:
11            messagebox.showerror("FTP Error", str(e))
12
13    threading.Thread(target=do_connect, daemon=True).start()
```

Listing 3.11: Koneksi ke FTP Server

3.7 Implementasi Game Multiplayer (game_window.py)

NetHub menyertakan game Tic-Tac-Toe multiplayer yang dapat dimainkan antar pengguna dalam room yang sama.

3.7.1 Logika Game

```
1 def check_win(self, p):
2     b = self.board
3     return ((b[0]==p and b[1]==p and b[2]==p) or
4             (b[3]==p and b[4]==p and b[5]==p) or
5             (b[6]==p and b[7]==p and b[8]==p) or
6             (b[0]==p and b[3]==p and b[6]==p) or
7             (b[1]==p and b[4]==p and b[7]==p) or
8             (b[2]==p and b[5]==p and b[8]==p) or
9             (b[0]==p and b[4]==p and b[8]==p) or
10            (b[2]==p and b[4]==p and b[6]==p))
```

Listing 3.12: Logika Pengecekan Kemenangan

3.7.2 Sinkronisasi State Game

Setiap gerakan pemain dikirim ke server dan di-broadcast ke semua pemain dalam room:

```
1 def on_click(self, index):
2     if self.board[index] == "":
3         self.send_func(f"GAME|MOVE|{index}|{self.turn}")
```

Listing 3.13: Pengiriman Gerakan Game

BAB 4

PENGUJIAN

Bab ini membahas pengujian yang dilakukan terhadap aplikasi NetHub untuk memastikan setiap fitur berfungsi dengan baik.

4.1 Pengujian Koneksi Server-Client

Pengujian pertama adalah memastikan server dapat berjalan dan menerima koneksi dari client.

4.1.1 Skenario Pengujian

1. Menjalankan server dengan perintah `python server.py`
2. Menjalankan client dengan perintah `python client_gui.py`
3. Memverifikasi koneksi berhasil terbentuk

4.1.2 Hasil Pengujian

Table 4.1: Hasil Pengujian Koneksi Server-Client

Test Case	Ekspektasi	Hasil
Server startup	Server menampilkan pesan listening	Berhasil
Client connect	Client terhubung tanpa error	Berhasil
Multiple clients	Server dapat menangani multiple client	Berhasil

4.2 Pengujian Autentikasi

4.2.1 Pengujian Register

Table 4.2: Hasil Pengujian Register

Test Case	Ekspektasi	Hasil
Register user baru	User tersimpan di users.json	Berhasil
Register username duplikat	Sistem menolak dengan pesan error	Berhasil
Password hashing	Password tersimpan dalam bentuk hash	Berhasil

4.2.2 Pengujian Login

Table 4.3: Hasil Pengujian Login

Test Case	Ekspektasi	Hasil
Login dengan kredensial valid	Login berhasil, masuk ke room selection	Berhasil
Login dengan password salah	Sistem menolak dengan pesan error	Berhasil
Login dengan username tidak ada	Sistem menolak dengan pesan error	Berhasil

4.3 Pengujian Fitur Chat

4.3.1 Pengujian Room System

Table 4.4: Hasil Pengujian Room System

Test Case	Ekspektasi	Hasil
Join room	User dapat masuk ke room yang dipilih	Berhasil
Room notification	User lain di room menerima notifikasi join	Berhasil
User list update	List user di room terupdate real-time	Berhasil

4.3.2 Pengujian Messaging

Table 4.5: Hasil Pengujian Messaging

Test Case	Ekspektasi	Hasil
Kirim pesan	Pesan terkirim dan tampil di semua client	Berhasil
Broadcast ke room	Pesan hanya diterima user di room yang sama	Berhasil
Emoji support	Emoji dapat dikirim dan ditampilkan	Berhasil

4.4 Pengujian Transfer File

4.4.1 Pengujian Upload File

Table 4.6: Hasil Pengujian Upload File

Test Case	Ekspektasi	Hasil
Upload file	File tersimpan di server_files	Berhasil
File notification	User di room menerima link download	Berhasil
Metadata saved	File metadata tersimpan dengan room info	Berhasil

4.4.2 Pengujian Download File

Table 4.7: Hasil Pengujian Download File

Test Case	Ekspektasi	Hasil
Download dari room yang sama	File dapat didownload	Berhasil
Download dari room berbeda	Akses ditolak	Berhasil
Download file tidak ada	Pesan error yang sesuai	Berhasil

4.5 Pengujian FTP Server

4.5.1 Pengujian Koneksi FTP

Table 4.8: Hasil Pengujian Koneksi FTP

Test Case	Ekspektasi	Hasil
FTP server startup	Server berjalan di port 2121	Berhasil
Login dengan kredensial NetHub	Login FTP berhasil	Berhasil
Anonymous access	Bisa akses folder public (read-only)	Berhasil

4.5.2 Pengujian Operasi File FTP

Table 4.9: Hasil Pengujian Operasi File FTP

Test Case	Ekspektasi	Hasil
Upload via FTP Browser	File terupload ke direktori user	Berhasil
Download via FTP Browser	File terdownload ke lokal	Berhasil
Delete file	File terhapus dari server	Berhasil
Share to chat	File dibagikan ke room chat	Berhasil

4.6 Pengujian Game Multiplayer

4.6.1 Pengujian Tic-Tac-Toe

Table 4.10: Hasil Pengujian Game Tic-Tac-Toe

Test Case	Ekspektasi	Hasil
Open game window	Game window terbuka dengan board kosong	Berhasil
Multiplayer sinkronisasi	Gerakan tersinkronisasi antar pemain	Berhasil
Win detection	Sistem mendeteksi pemenang dengan benar	Berhasil
Draw detection	Sistem mendeteksi hasil seri	Berhasil
Game reset	Board dapat direset untuk game baru	Berhasil

4.7 Pengujian Speed Test

Table 4.11: Hasil Pengujian Speed Test

Test Case	Ekspektasi	Hasil
Run speed test	Menampilkan dialog testing	Berhasil
Hasil download speed	Menampilkan kecepatan download dalam Mbps	Berhasil
Hasil upload speed	Menampilkan kecepatan upload dalam Mbps	Berhasil

4.8 Ringkasan Hasil Pengujian

Berdasarkan seluruh pengujian yang dilakukan, dapat disimpulkan bahwa:

1. Semua fitur utama aplikasi NetHub berfungsi sesuai dengan yang diharapkan.
2. Komunikasi client-server menggunakan TCP socket berjalan dengan stabil.
3. Sistem autentikasi terintegrasi antara chat server dan FTP server bekerja dengan baik.
4. Mekanisme broadcast untuk chat dan game multiplayer berfungsi dengan benar.
5. Transfer file baik melalui chat maupun FTP dapat dilakukan tanpa masalah.

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian aplikasi NetHub, dapat diambil kesimpulan sebagai berikut:

1. Implementasi TCP Socket Programming

Aplikasi NetHub berhasil mengimplementasikan komunikasi jaringan menggunakan TCP socket. Server dapat menangani multiple client secara bersamaan menggunakan multi-threading, dengan setiap client ditangani oleh thread terpisah untuk memastikan responsivitas sistem.

2. Sistem Chat Multi-User dengan Room

Fitur chat dengan sistem room berhasil diimplementasikan. Pengguna dapat bergabung ke room yang berbeda dan pesan hanya dikirim ke pengguna dalam room yang sama melalui mekanisme broadcast. Sistem juga menampilkan daftar pengguna aktif dalam setiap room.

3. Sistem Autentikasi Terintegrasi

Sistem autentikasi menggunakan hashing SHA-256 untuk keamanan password. Yang lebih penting, sistem autentikasi ini terintegrasi antara chat server dan FTP server, sehingga pengguna hanya perlu mendaftar sekali untuk mengakses kedua layanan.

4. Transfer File

Fitur transfer file berhasil diimplementasikan dalam dua cara:

- Transfer file dalam chat menggunakan encoding Base64
- Transfer file melalui FTP server dengan fitur lengkap (upload, download, delete)

5. FTP Server dengan pyftplib

FTP server yang terintegrasi dengan sistem autentikasi NetHub berhasil diimplementasikan menggunakan library `pyftplib`. Setiap pengguna

memiliki direktori pribadi dan admin memiliki akses ke seluruh direktori.

6. **Game Multiplayer**

Game Tic-Tac-Toe multiplayer berhasil diimplementasikan dengan sinkronisasi state antar pemain melalui server. Sistem dapat mendeteksi kemenangan, kekalahan, dan hasil seri.

7. **Protokol Komunikasi Custom**

Protokol komunikasi berbasis teks dengan format `COMMAND | PARAM1 | PARAM2 | . . .` terbukti efektif dan mudah untuk di-parse, memungkinkan penambahan fitur baru dengan mudah.

5.2 **Saran**

Untuk pengembangan lebih lanjut, berikut adalah beberapa saran yang dapat dipertimbangkan:

1. **Enkripsi End-to-End**

Implementasi enkripsi pada komunikasi client-server untuk meningkatkan keamanan, misalnya menggunakan TLS/SSL atau enkripsi asimetris.

2. **Database Relasional**

Mengganti penyimpanan JSON dengan database relasional seperti SQLite atau PostgreSQL untuk skalabilitas dan performa yang lebih baik.

3. **Fitur Private Message**

Menambahkan fitur pesan pribadi antar pengguna selain chat dalam room.

4. **History Chat**

Menyimpan riwayat chat sehingga pengguna yang baru bergabung dapat melihat pesan sebelumnya.

5. **Deployment ke Server Publik**

Mengembangkan aplikasi agar dapat di-deploy ke server publik dengan alamat IP yang dapat diakses dari internet.

6. **Mobile Application**

Mengembangkan versi mobile (Android/iOS) untuk meningkatkan aksesibilitas pengguna.

7. Notifikasi Push

Implementasi sistem notifikasi push untuk memberitahu pengguna tentang pesan baru.

8. Fitur Game Tambahan

Menambahkan game multiplayer lainnya seperti Chess, Checkers, atau game kartu.

9. Voice/Video Call

Integrasi fitur voice call atau video call menggunakan protokol seperti WebRTC.

10. Theme Customization

Memungkinkan pengguna untuk mengkustomisasi tampilan antarmuka sesuai preferensi.